Lab 2 : Empirical Analysis of Bubble Sort

Theory:

Bubble Sort is a very straightforward, comparison-based sorting algorithm. It works by repeatedly stepping through the list, comparing adjacent elements side-by-side, and swapping them if they are in the wrong order. This "bubbling" process is repeated until no swaps are needed during a full pass, which indicates that the list is finally sorted. While it is very easy to write and understand, it is highly inefficient for large datasets because of its $O(n^2)$ average and worst-case time complexity.

Algorithm:

```
void bubbleSort(vector<int>& arr) {

        int n = arr.size();

        bool swapped;



        for (int i = 0; i < n - 1; i++) {

          swapped = false;

        for (int j = 0; j < n - i - 1; j++) {

        if (arr[j] > arr[j + 1]) {

        swap(arr[j], arr[j + 1]);

           swapped = true;

        }

        }



        // If no two elements were swapped, then break early

        if (!swapped)

        break;

        }

}
```

Expressing Complexity of cases:

Case	Time Complexity	Condition	Comparisons	Swaps
Best	O(n)	Already sorted	n-1	0
Average	$O(n^2)$	Random order	$n^2/2$	$n^2/4$
Worst	$O(n^2)$	Reverse sorted	n(n-1)/2	n(n-1)/2

Source Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void generate(int *arr, int size, long long *count) {

        for (int i = 0; i < size; i++) { arr[i] = rand() % 1000; (*count)++; }

}

void generateINC(int *arr, int size, long long *count) {

        arr[0] = rand() % 5000; (*count)++;

        for (int i = 1; i < size; i++) { arr[i] = arr[i-1] + (rand() % 50 + 1); (*count) += 2; }

}

void generateDEC(int *arr, int size, long long *count) {

        arr[0] = rand() % 5000; (*count)++;

        for (int i = 1; i < size; i++) { arr[i] = arr[i-1] - (rand() % 50 + 1); (*count) += 2; }

}

void bubblesort(int *arr, int size, long long *count) {

        for (int i = 0; i < size - 1; i++) {

        for (int j = 0; j < size - i - 1; j++) {

        if (arr[j] > arr[j+1]) {

        int temp = arr[j];

        arr[j] = arr[j+1];

        arr[j+1] = temp;

        (*count) += 3;

    }     }}

}


void print(long long count) { printf("Count: %lld\n", count); }
```

```
int main() {
	srand(time(NULL));

	for (int size = 1000; size <= 10000; size += 1000) {
	printf("\n023bscit009_binesh\nFor input size:: %d", size);

	int *a = (int*)malloc(size * sizeof(int));
	int *b = (int*)malloc(size * sizeof(int));
	int *c = (int*)malloc(size * sizeof(int));
	long long count = 0;

	printf("\nAscending::\n"); count = 0;
	generateINC(b, size, &count); bubblesort(b, size, &count); print(count);

	printf("Random::\n"); count = 0;
	generate(a, size, &count); bubblesort(a, size, &count); print(count);

	printf("Descending::\n"); count = 0;
	generateDEC(c, size, &count); bubblesort(c, size, &count); print(count);

	free(a); free(b); free(c);
	}
	return 0;
}
```

Observations/Outputs:

Output 1 :

```
023bscit009_binesh
For input size:: 1000
Ascending::
Count: 1999
Random::
Count: 761611
Descending::
Count: 1500499
```

Output 2 :

```
023bscit009_binesh
For input size:: 5000
Ascending::
Count: 9999
Random::
Count: 18764030
Descending::
Count: 37502499
```

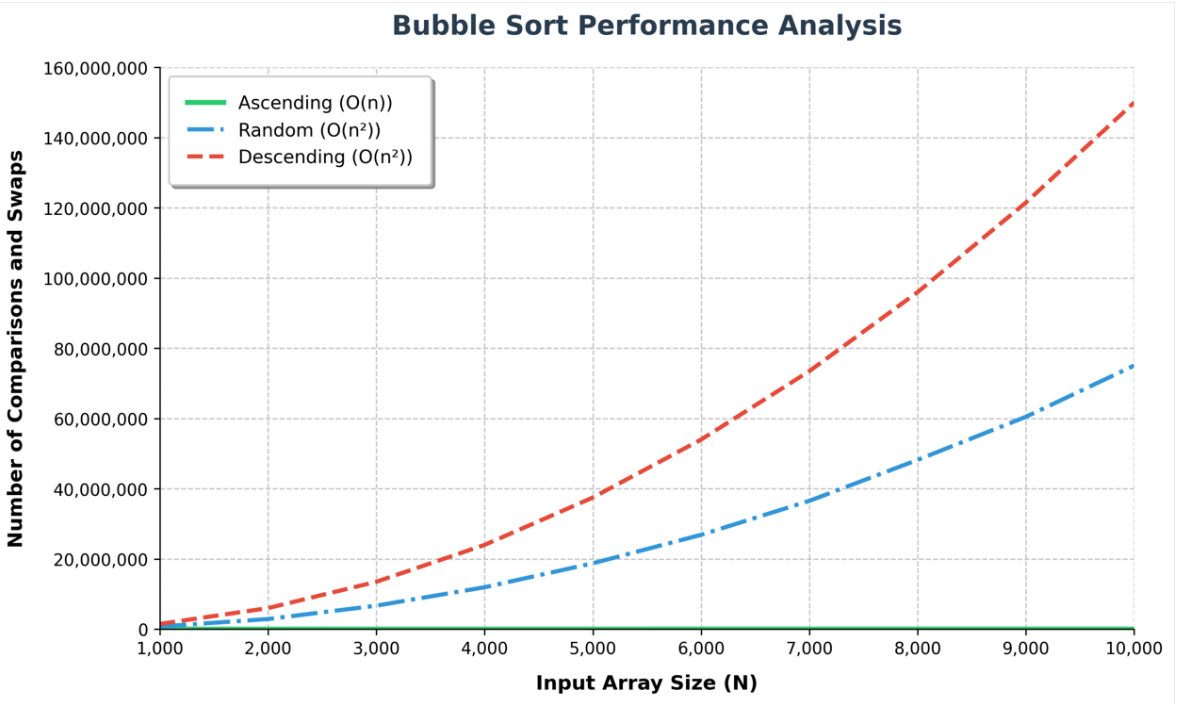
Output 3 :

```
023bscit009_binesh
For input size:: 10000
Ascending::
Count: 19999
Random::
Count: 75092179
Descending::
Count: 150004999
```

Data Table:

Input Size	Ascending	Random	Descending
1,000	1,999	761,611	1,500,499
2,000	3,999	2,894,261	6,000,999
3,000	5,999	6,694,914	13,501,499
4,000	7,999	11,943,262	24,001,999
5,000	9,999	18,764,030	37,502,499
6,000	11,999	26,877,027	54,002,999
7,000	13,999	36,551,149	73,503,499
8,000	15,999	48,221,975	96,003,999
9,000	17,999	60,485,634	121,504,499
10,000	19,999	75,092,179	150,004,999

Function Growth:



Conclusion:

This empirical analysis of Bubble Sort perfectly validates its theoretical performance characteristics across varying data distributions.

For the **Best Case (Ascending)**, it demonstrated linear growth $O(n)$ utilizing the "swapped" flag optimization, requiring minimal comparisons and zero swaps. However, for the **Average and Worst Cases (Random and Descending)**, it confirmed a drastic quadratic $O(n^2)$ growth rate. As predicted by the math, the operations increased exponentially as the input size grew, reaching completely maxed-out swap values when the list was reverse-sorted.

The key takeaway is that Bubble Sort is highly sensitive to the initial arrangement of data. While it can efficiently verify an already sorted list in just a single pass, its heavy $O(n^2)$ complexity makes it computationally expensive and strictly impractical for large-scale, unorganized datasets.